ICDFA GRC102 — Information Security Governance

Week 5 Linux Lab Assignment

Part 1: Setting Up a Simulated Environment

Student Name: Sumayyah Zubair
Course: GRC102 — Information Security Governance
Assignment: Simulating and Analyzing Security Governance Scenarios

1. Introduction

The objective of this practical exercise was to create a simulated security environment using Docker, perform vulnerability assessments, and implement a governance tracker for recording security policies, controls, risks, incidents, metrics, and audits. The exercise demonstrates how technical security weaknesses can be identified and documented to support security governance and risk management.

2. Environment Setup

The lab environment was configured using Docker Compose on Kali Linux. Three containers were deployed and connected through a Docker network named `secgov_network`.

The containers were:

- **web_server:** Runs Apache Struts 2.3.30 to simulate a vulnerable web application.
- **database_server:** Runs MySQL 5.7 and stores sample customer records.
- **monitoring_server:** Runs Ubuntu 20.04 for monitoring-related activities.

The web application was mapped to `127.0.0.1:8080`, restricting published access to the local machine. The database was initialized with a `customers` table containing three sample records.

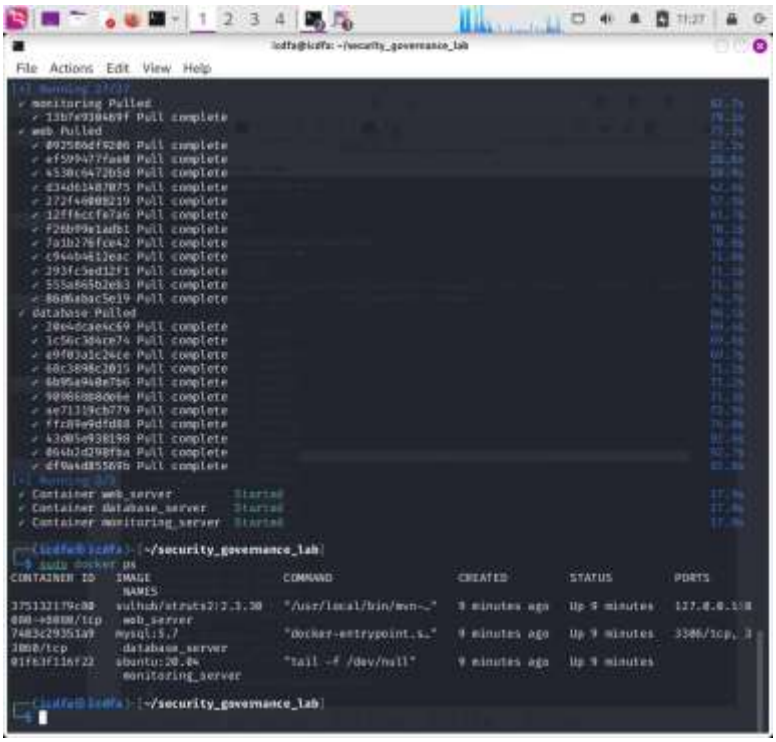


Figure 1: Docker containers running in the simulated security environment.

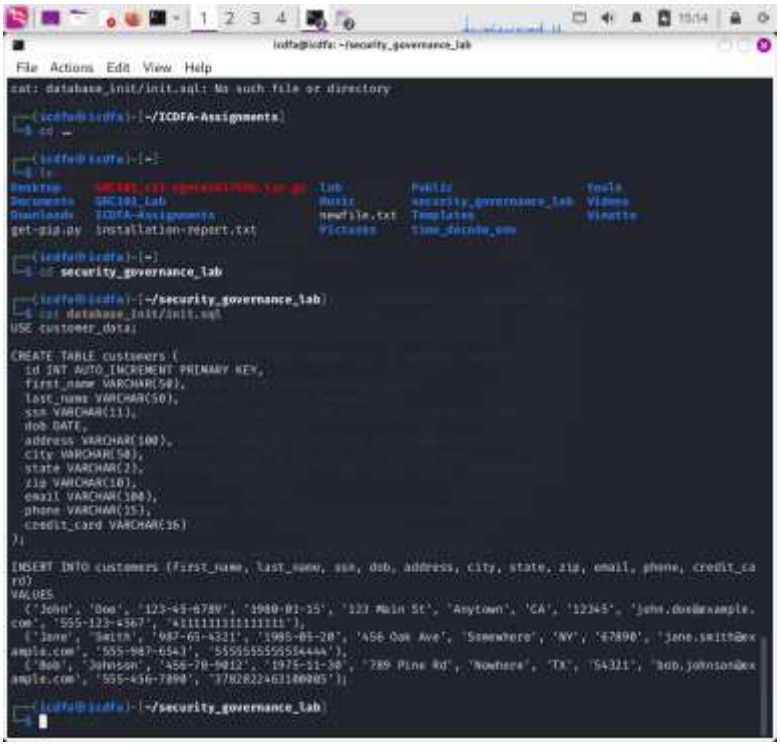


Figure 2: Verification of the customers table and three sample records in MySQL.

3. Vulnerability Assessment

A Python vulnerability scanner was created to perform three checks: Apache Struts vulnerability detection, MySQL configuration assessment, and network segmentation assessment. The scanner generated a timestamped JSON report containing the results.

The latest scan produced the following findings:

Security check	Result	Severity
Apache Struts S2-045	Vulnerable indicator detected	Critical
MySQL configuration	Default credentials accepted	High
Network segmentation	Database reachable from web container	Medium

The scan recorded one critical finding, one high-severity finding, and one medium-severity finding. The results indicate that the application vulnerability, database credentials, and network access should be reviewed and remediated.

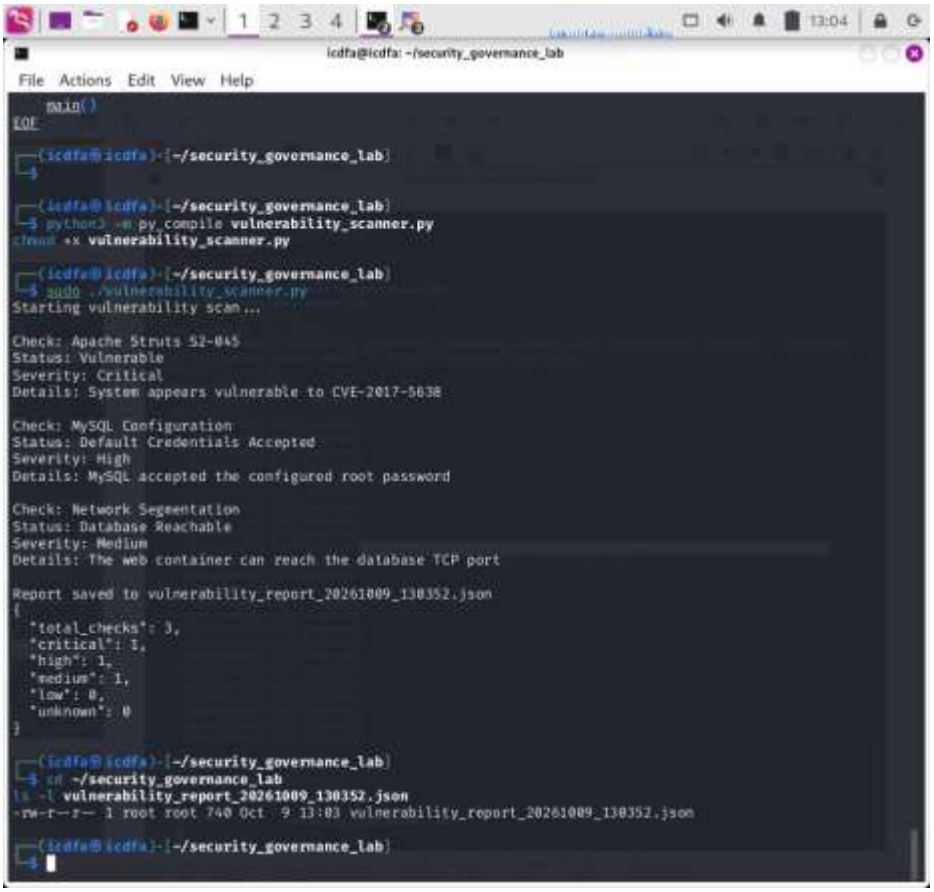


Figure 3: Vulnerability scanner results showing the three security findings.

The generated report was saved as `vulnerability_report_20261009_130352.json`.

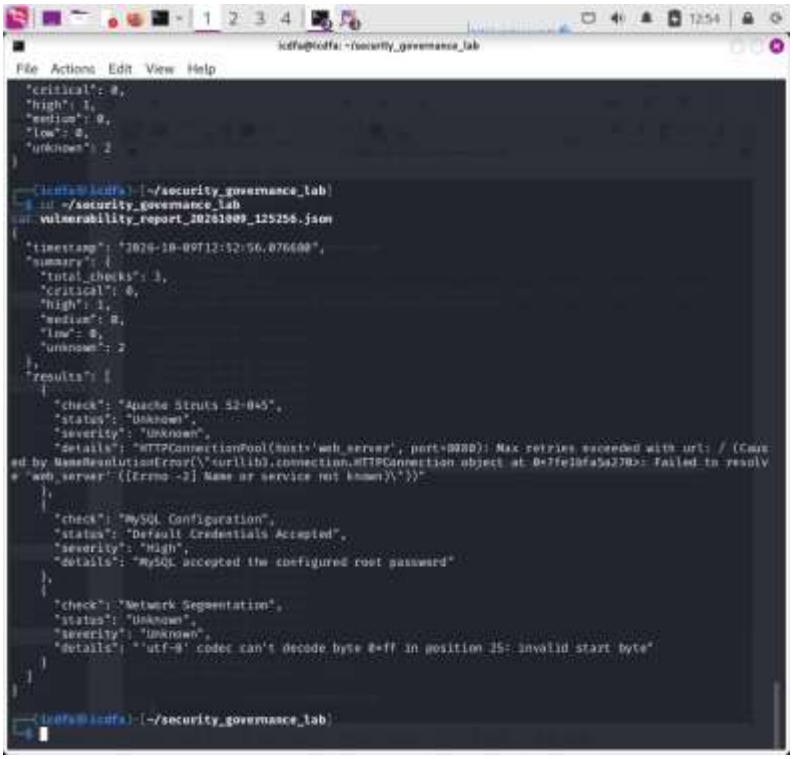


Figure 4: Generated JSON vulnerability assessment report.

4. Patch Management

A Python patch-management script was developed to list three simulated remediation items:

1. `PATCH-001` — Apache Struts2 S2-045 Patch, Critical.
2. `PATCH-002` — MySQL Security Configuration, High.
3. `PATCH-003` — Network Segmentation Rules, Medium.

The script supports listing available patches and simulating the application of a selected patch. The simulation records patch status in a JSON file; it does not actually patch the running containers.

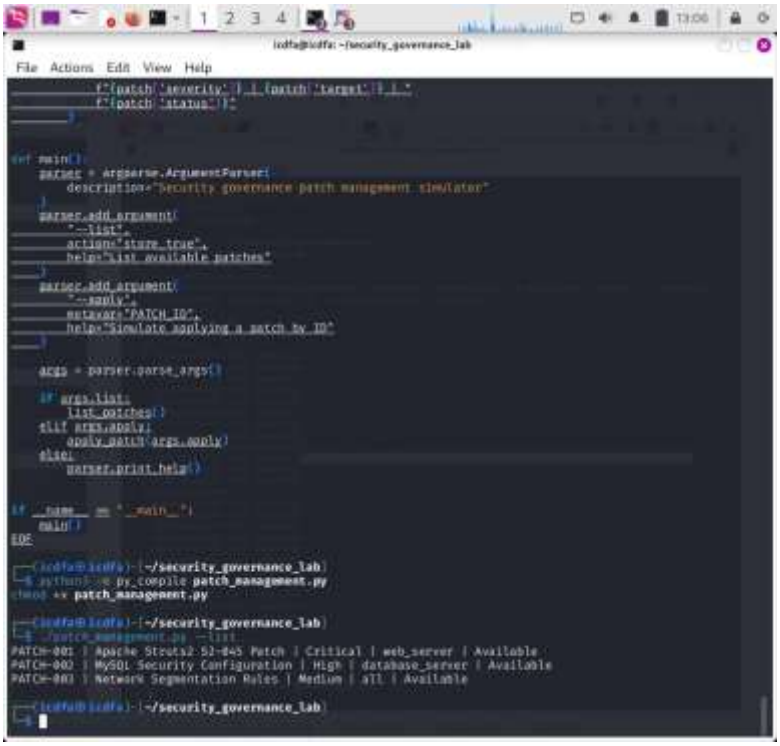


Figure 5: Available patches and their severity levels and targets.

5. Security Governance Tracker

A governance tracker was implemented in Python to record policies, controls, risks, incidents, metrics, and audits. Sample governance data was initialized, and a dashboard was generated to summarize the records.

The dashboard showed:

Governance category	Records
Policies	3
Controls	3
Risks	3
Incidents	1
Metrics	3
Audits	1

The sample metrics showed patch compliance with a target of 95 and an actual value of 85, vulnerability remediation time with a target of 7 and an actual value of 12, and security incidents with a target of 0 and an actual value of 1. These results highlight areas that require management attention.

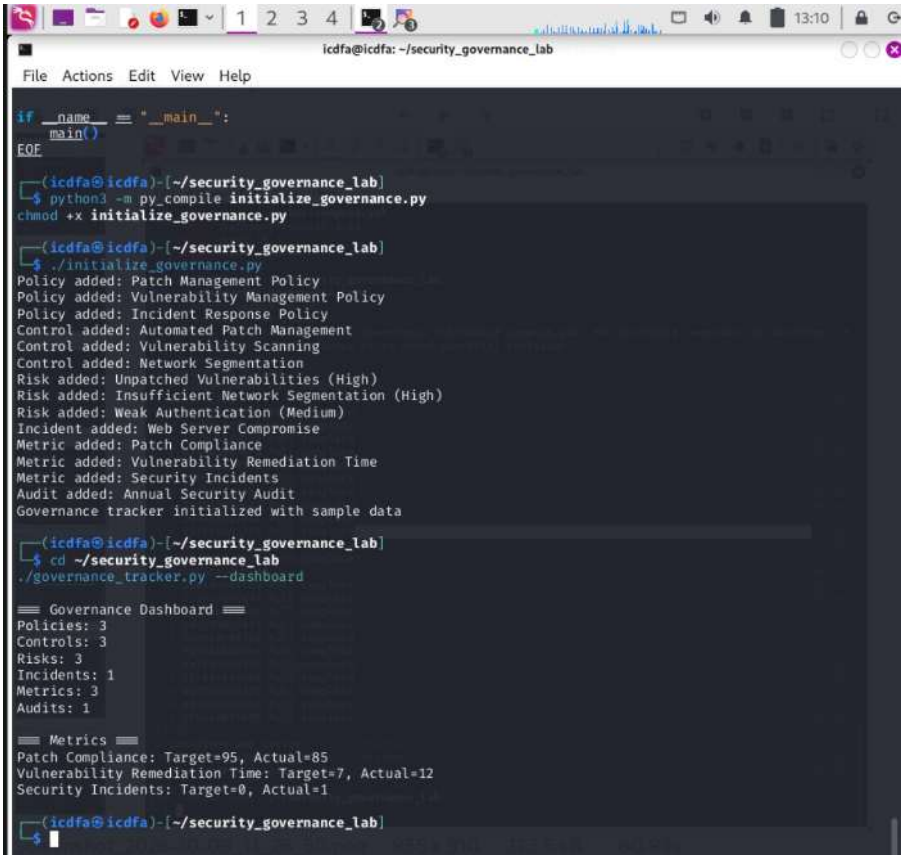


Figure 6: Governance tracker dashboard displaying policies, controls, risks, incidents, metrics, and audits.

6. Comparison with the Equifax Data Breach

The lab demonstrates security governance lessons associated with the 2017 Equifax data breach, which involved the exploitation of a known Apache Struts vulnerability. The simulated vulnerable web application represents the risk of leaving known software vulnerabilities unaddressed. The vulnerability scanner illustrates how security teams can identify and report weaknesses, while the patch-management component demonstrates the need to track remediation activities and assign accountability.

The database and network findings also illustrate why sensitive information must be protected through strong authentication, appropriate access restrictions, and effective network segmentation. The governance tracker links technical findings to policies, controls, risks, incidents, metrics, and audits. In a real organisation, these governance processes help management monitor remediation, enforce accountability, and reduce the likelihood and impact of security incidents. This lab is a simplified educational simulation and does not reproduce every circumstance of the Equifax breach.

7. Conclusion

Part 1 established a Docker-based security lab, initialized a MySQL database, performed vulnerability checks, implemented a patch-management simulator, and created a governance tracker. The assessment identified critical, high, and medium-severity findings. The governance dashboard summarized sample governance records and highlighted performance gaps. These activities demonstrate how technical assessments and governance tracking can work together to support information security risk management.

Part 2 — Analyzing Security Governance Failures

2.1 Attack Scenario Simulation

The `simulate_attack.py` script was created to simulate an attack scenario involving the Apache Struts web application. The script sent a normal HTTP request to the local application at `http://127.0.0.1:8080` and received an HTTP status of 200.

The script generated an `attack_log.json` file recording the simulated scenario. The log indicated that the simulation was not a successful exploitation, no customer data was accessed, and zero records were affected.

This exercise demonstrated how an attack scenario can be documented for governance analysis. However, it did not prove that the vulnerability was successfully exploited.

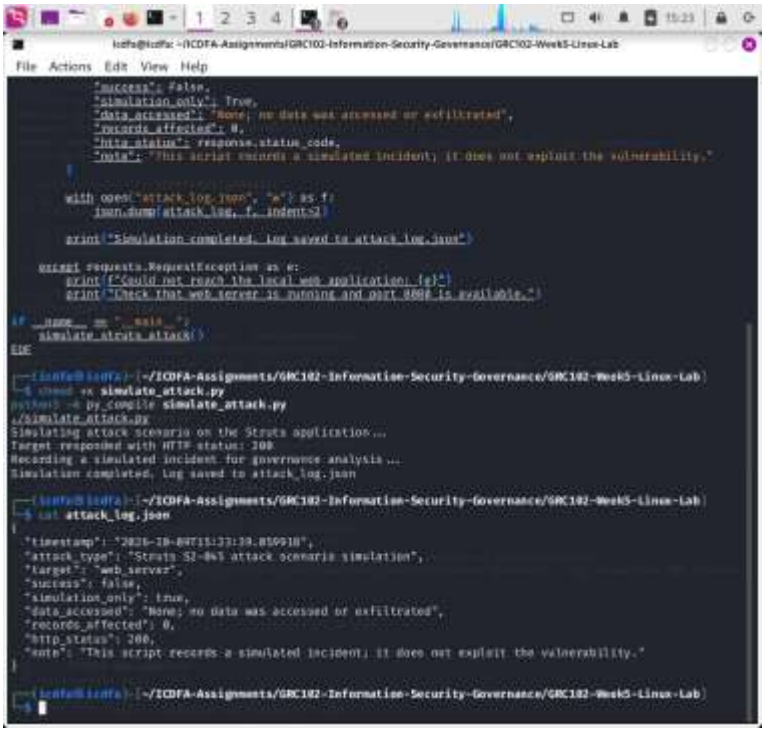


Figure 7: Attack simulation output and the contents of attack_log.json.

2.2 Governance Failure Analysis

The `analyze_governance_failures.py` script was developed to analyze potential governance weaknesses using available lab information. It generated the report `governance_failure_report_20261009_152447.json`.

The analysis identified four preliminary findings:

Governance category	Potential issue	Recommended action
Incident Detection	Monitoring and alerting effectiveness has not been verified.	Implement monitoring, alerting, and tested incident-response procedures.
Policy Implementation	Patch management may not be consistently implemented.	Assign owners and deadlines, and verify remediation.
Security Architecture	The web container can reach the database TCP port.	Restrict communication between services to required connections.
Authentication and Access Control	The configured MySQL root password was accepted.	Use strong credentials and least-privilege database accounts.

The script displayed a warning that `patch_status.json` was not found and therefore used default patch-status values. Consequently, the findings should be treated as preliminary indicators rather than independently verified control failures.

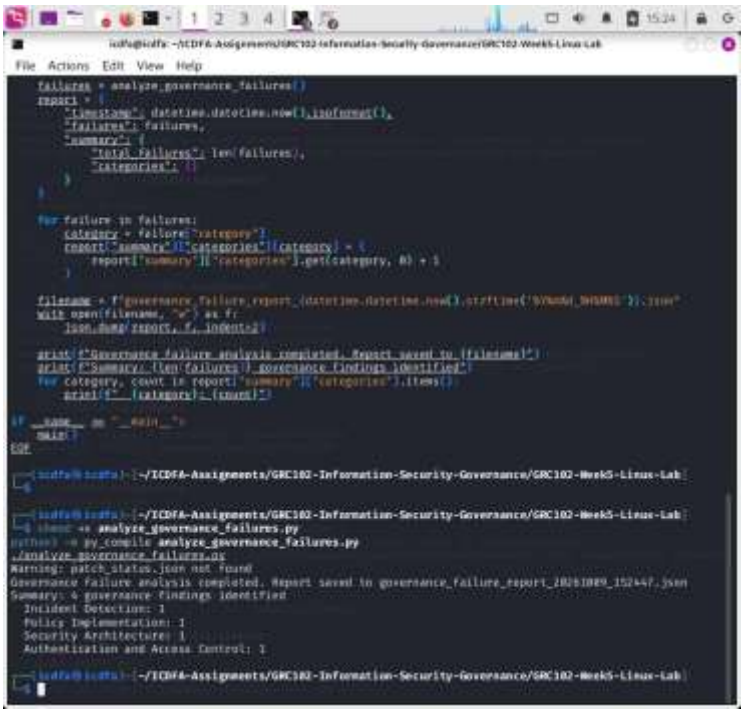


Figure 8: Governance failure analysis output showing the four findings.

2.3 Comparison with the Equifax Data Breach

The 2017 Equifax data breach involved the exploitation of a known vulnerability in Apache Struts. The incident demonstrated the consequences that can arise when a critical software vulnerability is not remediated effectively.

The simulated lab reflects this class of risk by using an older Struts application and reporting a critical vulnerability indicator. The vulnerability scanner illustrates how security teams can identify and document potential weaknesses, while the patch-management component demonstrates the importance of tracking remediation activities.

The database and network findings also illustrate why sensitive information must be protected through strong authentication, appropriate access restrictions, and effective network segmentation. Weak credentials and excessive connectivity can increase the potential impact of a compromised application.

However, this lab is a simplified educational simulation. It does not reproduce the full Equifax incident, and its attack simulation did not demonstrate successful exploitation or access to customer data.

2.4 Key Governance Lessons Learned

The exercise highlighted the following governance lessons:

1. Vulnerability management: Organisations should maintain an inventory of systems, monitor security advisories, prioritise critical vulnerabilities, and verify remediation.
2. Accountability: Security policies require responsible owners, deadlines, and evidence that corrective actions have been completed.
3. Access control: Strong credentials and least-privilege database accounts help reduce unauthorised access.
4. Network segmentation: Restricting communication between application and database services can limit the potential impact of a compromised component.
5. Incident response: Organisations should maintain monitoring, escalation procedures, evidence collection, and tested response plans.
6. Assurance and metrics: Audits and measurable indicators help management determine whether security controls operate effectively rather than merely exist as written policies.

3. Implementation and Verification of Security Governance Controls

3.1 Objective

The objective of this exercise was to implement and verify security governance controls using Python scripts in the Kali Linux environment. The exercise focused on establishing governance policies, implementing security controls, defining security metrics, and verifying that the required governance artefacts were recorded.

3.2 Implementation of Governance Controls

The `implement_governance_controls.py` script was used to implement the governance controls. The implementation included recording security policies, controls, and metrics using the governance tracking system.

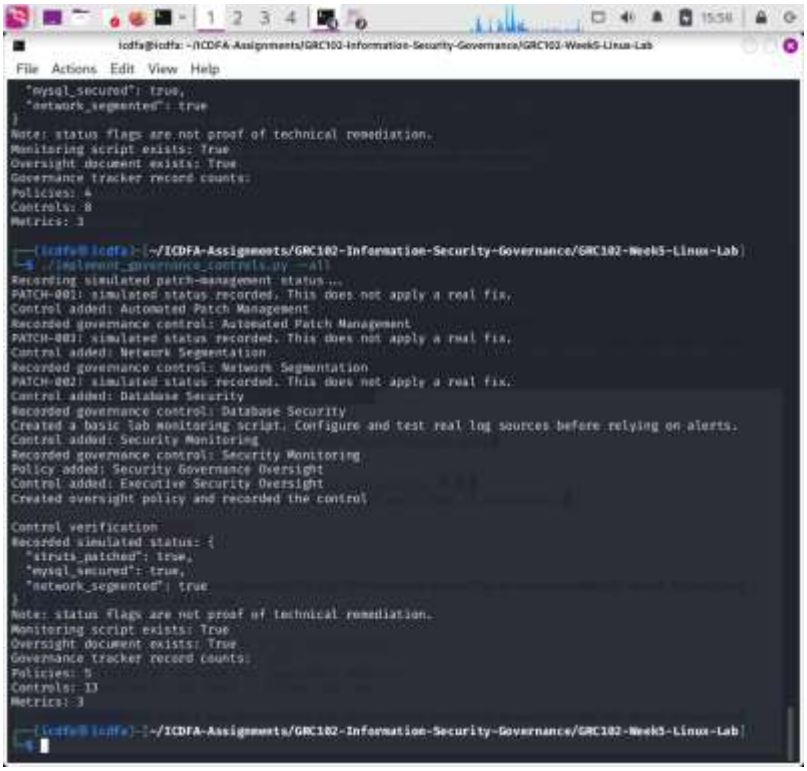
The governance activities covered security oversight, policy implementation, control monitoring, and verification of the required governance records. The exercise also included a monitoring script and a governance oversight document.

3.3 Verification Results

The verification process confirmed that the governance tracker contained the following records:

- **Security policies:** 5
- **Security controls:** 13
- **Security metrics:** 3

- **Monitoring script:** Present
- **Governance oversight document:** Present



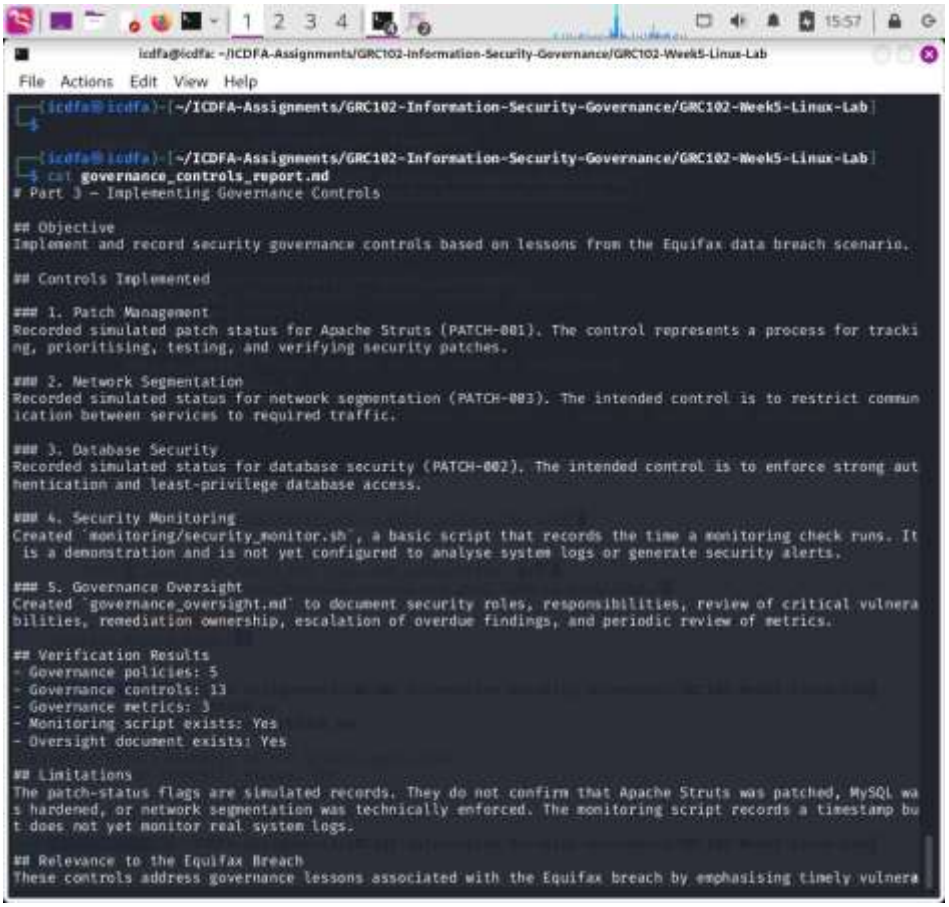
The verification results indicate that the expected governance records and supporting files were available in the lab environment.

The exercise also included simulated patch-management status checks. These checks demonstrate the use of a governance process to track security-related activities. However, the simulated status flags do not independently prove that vulnerabilities were technically remediated.

3.4 Relevance to Security Governance

Implementing and verifying governance controls helps an organisation establish accountability and demonstrate that security policies and procedures have been documented. Maintaining records of policies, controls, and metrics also provides a foundation for monitoring compliance and reporting security performance to management.

This exercise demonstrates the importance of verifying governance records rather than assuming that controls are implemented simply because documentation exists.



3.5 Conclusion

The governance-control implementation and verification exercise completed successfully. The tracker contained five policies, thirteen controls, and three metrics, while the monitoring script and governance oversight document were present. Further technical testing would be required to confirm the operational effectiveness of the controls.

4. Security Governance Metrics and Executive Reporting

4.1 Objective

The objective of this exercise was to generate security governance metrics, visualise security performance trends, and prepare reports suitable for management and leadership review. The exercise used Python to generate historical sample metrics, create charts, and produce an executive dashboard and a board report.

4.2 Generation of Security Governance Metrics

The `governance_metrics.py` script was executed successfully. It generated illustrative metrics covering six months, from May to October 2026.

The metrics included:

- **Patch compliance:** Measures the proportion of relevant systems or patches that meet the defined patching requirements.
- **Vulnerability remediation time:** Measures how long it takes to remediate identified vulnerabilities.
- **Security incidents:** Tracks the number of recorded security incidents during the reporting period.
- **Security monitoring coverage:** Represents the proportion of relevant systems or activities covered by security monitoring.
- **Policy compliance:** Measures adherence to established security policies.

The generated data was used to demonstrate how security performance can be monitored against defined targets. These figures were illustrative sample data and were not measurements collected from a production environment.

4.3 Security Metrics Visualisations

The script generated five visualisation files in the `metrics_visualizations/` directory:

1. `patch_compliance.png`
2. `vuln_remediation.png`
3. `security_incidents.png`
4. `monitoring_coverage.png`
5. `policy_compliance.png`

The charts display the sample metrics over time and include target lines for comparison. This approach makes it easier to identify trends, compare performance against targets, and highlight areas that may require further investigation.

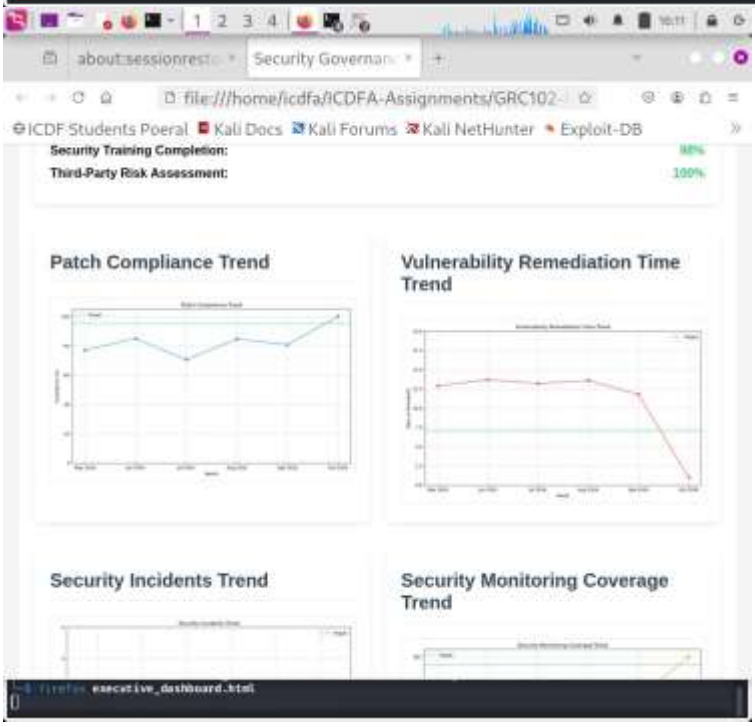
4.4 Executive Dashboard

The script generated `executive_dashboard.html`, which was opened successfully in Firefox. The dashboard presented a security posture summary, risk summary, compliance summary, five trend charts, and key recommendations.

The dashboard included illustrative values such as 100% patch compliance, one day for vulnerability remediation, zero current-month security incidents, 100% security monitoring coverage, and 100% policy compliance. These figures demonstrate how governance information can be presented in a consolidated and accessible format.

The dashboard also displayed sample risk and compliance figures. These values should be treated as examples rather than verified organisational results.



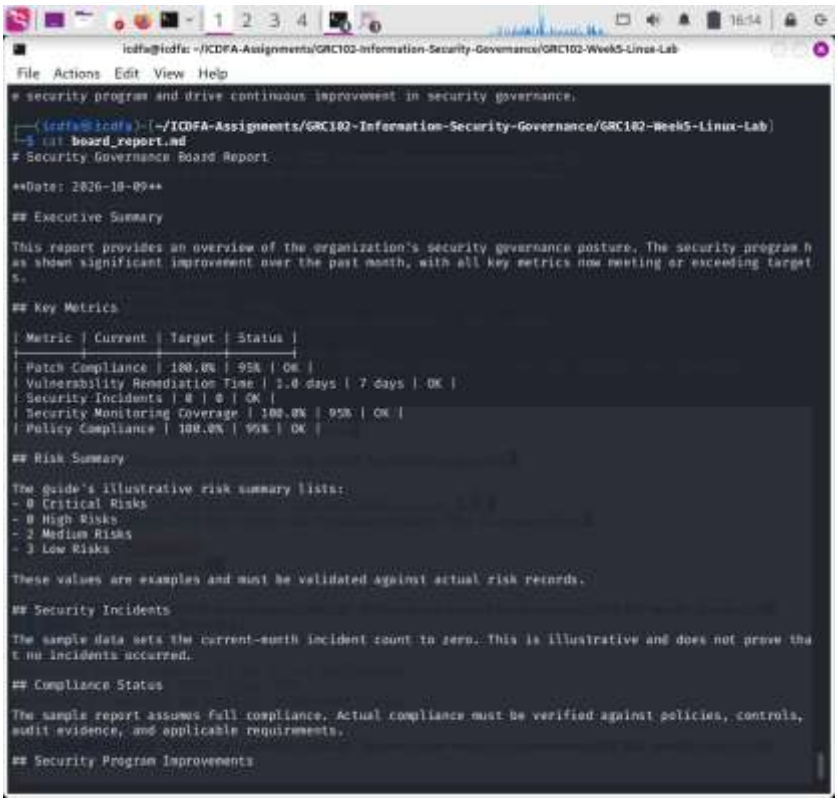


4.5 Board Report

The script also generated `board_report.md`, a Markdown report containing an executive summary, key metrics, a risk summary, security incident information, compliance status, security programme improvements, recommendations, and a conclusion.

The report demonstrates how technical security information can be summarised for leadership. Its recommendations include improving patch management, strengthening network segmentation and database security, enhancing security monitoring, improving governance oversight, and reviewing security performance regularly.

The report notes that the illustrative risk and compliance figures must be validated against actual risk records, controls, audit evidence, and applicable requirements before being used for formal decision-making.



4.6 How Security Metrics Support Governance

Security metrics support governance by providing measurable indicators that help management assess security performance, monitor progress towards established targets, and identify areas requiring corrective action. For example, patch compliance can help management evaluate patch-management performance, while vulnerability remediation time can indicate whether identified vulnerabilities are being addressed promptly.

Metrics also improve accountability and reporting by giving management a consistent way to review security activities over time. When supported by reliable operational data, documented targets, and evidence of control effectiveness, these metrics can inform risk-based decisions, prioritise resources, support compliance reviews, and encourage continuous improvement.

4.7 Conclusion

The metrics-generation and reporting exercise completed successfully. The Python script generated five visualisations, an executive dashboard, and a board report. The dashboard opened correctly and displayed the metrics, trend charts, and recommendations.

The exercise demonstrated how security governance metrics can be transformed into visual information and leadership reports. Since the generated values were illustrative, actual system data, risk records, and audit evidence would be required to establish the organisation's real security posture and control effectiveness.

LINK TO GITHUB REPO FOR EVIDENCE: https://github.com/Sumayyah-zubair/ICDFA-Assignments/tree/main/GRC102-Information-Security-Governance/GRC102-Week5-Linux-Lab